UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

INFORME TÉCNICO Y EXPLICATIVO DEL PROYECTO

"SISTEMA DE CONTROL DE INVENTARIO – FOLIO"

INTEGRANTES:

- QUISPE TICA Ruth Nayeli

DOCENTE:

- FERNANDEZ BEJARANO, Raul Enrique

CICLO: IV
SECCIÓN: A1

FECHA: 9 de noviembre del 2026

HUANCAYO - PERÚ
2026

1. INTRODUCCIÓN

El presente informe tiene como finalidad explicar y sustentar técnica y funcionalmente el proyecto denominado **Sistema de Control de Inventario – Folio**, una aplicación web desarrollada para facilitar la administración y el control de productos dentro de un almacén o negocio.

El sistema permite registrar productos, modificar información, controlar las cantidades disponibles, registrar entradas y salidas, administrar categorías y generar informes que facilitan la toma de decisiones.

Para desarrollar esta aplicación se utilizaron React, JavaScript, Vite y CSS3. Estas tecnologías permiten construir una interfaz interactiva, organizar el código mediante componentes y realizar operaciones automáticas sobre los datos del inventario.

El proyecto busca reducir los errores que pueden producirse al llevar un control manual de los productos y proporcionar información actualizada sobre las existencias y el valor económico del inventario.

2. PLANTEAMIENTO DEL PROBLEMA

En muchos negocios, el control de productos se realiza mediante registros manuales o documentos que no siempre se actualizan correctamente.

Esta situación puede ocasionar problemas como:

- Desconocimiento de la cantidad real de productos disponibles.
- Dificultad para identificar productos agotados.
- Falta de control sobre las entradas y salidas.
- Problemas al calcular el valor económico del inventario.
- Dificultad para identificar cuándo es necesario comprar nuevos productos.

Ante esta problemática, se desarrolló el Sistema de Control de Inventario – Folio, que permite organizar los productos y controlar sus existencias desde una sola aplicación web.

3. OBJETIVOS

3.1. Objetivo general

Desarrollar una aplicación web de control de inventario utilizando React y JavaScript, que permita registrar productos, controlar existencias, administrar movimientos y generar informes para mejorar la organización del almacén.

3.2. Objetivos específicos

- Registrar productos con su código, nombre, categoría, precio y cantidad.
- Permitir editar y eliminar productos registrados.
- Controlar las entradas y salidas de existencias.
- Evitar que las cantidades disponibles sean negativas.

- Identificar productos con stock bajo o agotado.
- Administrar las categorías de los productos.
- Calcular el valor económico del inventario.
- Generar informes y exportar información en formato CSV.

4. FUNDAMENTO TECNOLÓGICO

Para el desarrollo del proyecto se utilizaron las siguientes tecnologías:

4.1. React

React es una biblioteca de JavaScript utilizada para construir interfaces de usuario mediante componentes reutilizables.

En este proyecto permite dividir la aplicación en diferentes módulos, como Inventario, Movimientos, Categorías, Informes y Centro de Ayuda.

Sustento técnico:

En el archivo `src/App.jsx` se utiliza el hook `useState()` para administrar los productos, categorías, movimientos y la vista seleccionada.

Ejemplo:

```
const [productos, setProductos] = useState(initialProducts)
```

Esta instrucción crea un estado que contiene los productos iniciales y permite actualizar la información cuando se registra, edita o elimina un producto.

4.2. JavaScript

JavaScript es el lenguaje de programación que permite desarrollar las operaciones y la lógica del sistema.

Se utiliza para:

- Registrar productos.
- Validar los datos ingresados.
- Actualizar las existencias.
- Calcular el valor del inventario.
- Determinar el estado de los productos.
- Registrar los movimientos.
- Generar archivos para exportar información.

En el archivo `src/utils/inventory.js` se encuentran funciones como `valorTotal()`, `unidades()`, `estado()` y `requierenAtencion()`, que permiten realizar los cálculos y controles principales del inventario.

4.3. Vite

Vite es una herramienta que permite ejecutar el entorno de desarrollo de la aplicación.

Se utiliza para iniciar el servidor local y visualizar el sistema durante su desarrollo.

Sustento técnico:

En el archivo `package.json` se encuentra configurado el comando:

```
"dev": "vite"
```

Esto permite iniciar el proyecto mediante `npm run dev`.

4.4. CSS3

CSS3 permite diseñar y organizar la apariencia visual de la aplicación.

Se utiliza para establecer colores, tamaños, distribución de elementos, tablas, formularios, botones, indicadores y diferentes secciones de la interfaz.

El archivo `src/styles.css` contiene las reglas de diseño que permiten mantener una presentación uniforme y organizada.

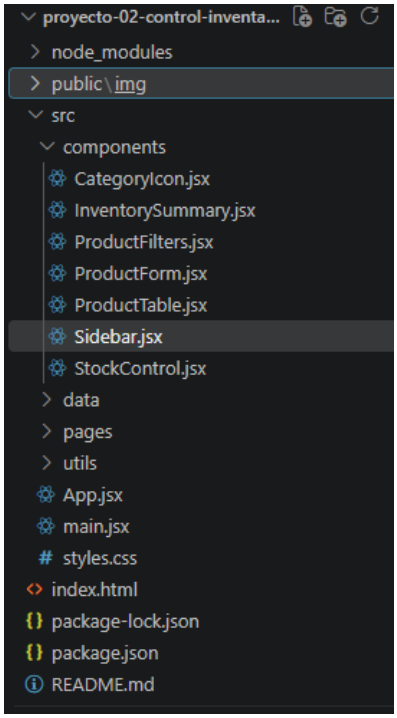
4.5. Lucide React

Es una biblioteca de iconos utilizada para representar visualmente las acciones del sistema.

Por ejemplo, se utilizan iconos para agregar productos, editar registros, eliminar información, descargar informes y controlar movimientos.

5. ESTRUCTURA Y ORGANIZACIÓN DEL PROYECTO

El proyecto está organizado en carpetas y archivos que cumplen funciones específicas.



5.1. Carpeta components

Contiene los componentes reutilizables:

- `ProductForm.jsx`: formulario para registrar y editar productos.
- `InventorySummary.jsx`: muestra el resumen general del inventario.
- `CategoryIcon.jsx`: presenta iconos según la categoría.
- `ProductTable.jsx`: muestra los productos en una tabla.
- `StockControl.jsx`: permite registrar entradas y salidas.
- `Sidebar.jsx`: contiene el menú de navegación.
- `ProductFilters.jsx`: permite buscar y filtrar productos.

5.2. Carpeta pages

Contiene los módulos principales:

- `Inventario.jsx`: administra los productos y sus existencias.
- `Movimientos.jsx`: presenta el historial de entradas y salidas.
- `Categorias.jsx`: administra las categorías.
- `Informes.jsx`: presenta estadísticas y permite descargar informes.
- `Ayuda.jsx`: proporciona instrucciones para utilizar el sistema.

5.3. Carpeta data

Contiene el archivo `initialProducts.js`, donde se encuentran los datos iniciales de los productos utilizados para iniciar la aplicación.

5.4. Carpeta utils

Contiene el archivo `inventory.js`, que almacena funciones auxiliares para calcular valores, cantidades, estados de productos y generar archivos CSV.

5.5. Archivos principales

- `App.jsx`: componente principal que administra los datos y conecta los diferentes módulos.
- `main.jsx`: punto de entrada de la aplicación React.
- `styles.css`: contiene los estilos visuales.
- `package.json`: contiene las dependencias y los comandos de ejecución del proyecto.

6. DESARROLLO Y SUSTENTACIÓN DE LOS MÓDULOS

6.1. MÓDULO DE INVENTARIO

Archivo principal: `src/pages/Inventario.jsx`.

Este módulo es el encargado de administrar los productos registrados en el sistema.

Sus principales funcionalidades son:

- Registrar nuevos productos.
- Mostrar los productos en una tabla.
- Buscar productos por nombre o código.
- Filtrar por categoría.
- Filtrar por estado.
- Editar información.
- Eliminar productos.
- Ajustar las cantidades disponibles.
- Mostrar alertas de productos que necesitan atención.

Sustento técnico:

En este módulo se utiliza el método `filter()` para mostrar los productos que coinciden con la búsqueda y los filtros seleccionados.

También se utiliza `useMemo()` para memorizar el resultado del filtrado y evitar repetir ese cálculo cuando sus datos de entrada no cambian.

El componente `ProductTable.jsx` presenta los productos mediante una tabla, mientras que `InventorySummary.jsx` muestra los indicadores generales.

Importancia: Permite conocer qué productos existen, cuántas unidades hay disponibles y cuáles necesitan reposición.

6.2. MÓDULO DE REGISTRO Y EDICIÓN DE PRODUCTOS

Archivo principal: `src/components/ProductForm.jsx`.

Este módulo permite ingresar los datos de cada producto.

Los campos principales son:

- Código del producto.
- Categoría.
- Nombre.
- Descripción.
- Imagen.
- Precio unitario.
- Stock actual.
- Stock mínimo.

Se utiliza `useState()` para almacenar temporalmente los datos que el usuario escribe en el formulario.

La función `validar()` comprueba que los campos obligatorios estén completos y que los valores numéricos sean correctos.

Por ejemplo:

- El código no puede estar vacío.
- No se permiten códigos duplicados.
- El precio debe ser mayor o igual a cero.
- El stock debe ser un número entero no negativo.
- El stock mínimo debe ser un número entero no negativo.

Para evitar duplicados se utiliza el método `some()`, que comprueba si ya existe otro producto con el mismo código.

También se utiliza `FileReader` para cargar imágenes y convertirlas en una dirección de datos que puede mostrarse en la vista previa.

Importancia: Permite mantener un catálogo de productos organizado y reducir errores al registrar información.

6.3. MÓDULO DE CONTROL DE EXISTENCIAS

Archivo principal: `src/components/StockControl.jsx`.

Este módulo permite modificar las cantidades disponibles de un producto mediante dos operaciones:

Entrada: Incrementa la cantidad de productos disponibles.

Salida: Disminuye la cantidad de productos disponibles.

El usuario selecciona un producto, elige el tipo de operación e indica la cantidad que desea agregar o retirar.

Se utiliza la variable `delta` para determinar el cambio en el stock.

- Si es una entrada, la cantidad se suma.
- Si es una salida, la cantidad se resta.

La variable `resultante` permite calcular cuántas unidades quedarán después de la operación.

Antes de confirmar, el sistema valida que la cantidad sea un número entero mayor que cero y que la salida no supere las existencias disponibles.

En `App.jsx`, la función `cambiarStock()` actualiza el inventario utilizando `map()` y evita que el stock sea negativo mediante `Math.max(0, p.stock + delta)`.

Importancia: Permite mantener un control adecuado de las existencias y evitar errores en las cantidades registradas.

6.4. MÓDULO DE MOVIMIENTOS

Archivo principal: `src/pages/Movimientos.jsx`.

Este módulo registra y muestra el historial de las operaciones realizadas sobre las existencias.

Presenta información como:

- Fecha y hora.
- Código del producto.
- Nombre del producto.
- Tipo de movimiento.
- Cantidad ingresada o retirada.
- Stock resultante.
- Origen del movimiento.

En `App.jsx` se encuentra la función `anotar()`, que registra los movimientos en el estado de React.

Cada movimiento contiene un identificador, fecha, producto, tipo, cantidad, stock final y origen.

En `Movimientos.jsx` se utiliza `filter()` para buscar movimientos por producto o tipo de operación.

También se utiliza `reduce()` para calcular la cantidad total de unidades que ingresaron y salieron.

Importancia: Permite conocer el historial de cambios y mantener una trazabilidad de las operaciones realizadas durante la sesión.

6.5. MÓDULO DE CATEGORÍAS

Archivo principal: `src/pages/Categorias.jsx`.

Este módulo permite organizar los productos según su clasificación.

Las funciones principales son:

- Crear nuevas categorías.
- Cambiar el nombre de una categoría.
- Eliminar categorías que no tengan productos asociados.
- Consultar cuántos productos pertenecen a cada categoría.
- Mostrar las unidades y el valor económico por categoría.

En `App.jsx` se implementan las funciones `agregarCategoria()`, `renombrarCategoria()` y `eliminarCategoria()`.

Para comprobar si una categoría ya existe se utiliza `some()`.

Cuando se cambia el nombre de una categoría, se utiliza `map()` para actualizar también la categoría de los productos relacionados.

Asimismo, se impide eliminar una categoría cuando todavía contiene productos registrados.

Importancia: Facilita la organización del catálogo y permite consultar los productos de acuerdo con su clasificación.

6.6. MÓDULO DE INFORMES

Archivo principal: `src/pages/Informes.jsx`.

Este módulo presenta información estadística y económica del inventario.

Sus principales funciones son:

- Mostrar el valor total del inventario.
- Contabilizar los productos registrados.
- Mostrar las unidades disponibles.
- Calcular el costo estimado de reposición.
- Mostrar el valor de los productos por categoría.
- Identificar los productos de mayor valor.
- Presentar una lista de productos que necesitan reposición.
- Descargar un informe en formato CSV.

Se utiliza la función `valorTotal()` para calcular el valor económico de todos los productos.

La operación aplicada es:

Valor del inventario = Precio unitario × Cantidad disponible.

Posteriormente, se suman los resultados de todos los productos.

Para identificar los productos más valiosos se utiliza `sort()` y se ordenan según su valor total.

Para calcular el costo de reposición se determina cuántas unidades faltan para alcanzar el stock mínimo y se multiplica esa cantidad por el precio unitario.

Finalmente, la función `descargarCSV()` genera un archivo que puede abrirse en programas como Microsoft Excel.

Importancia: Proporciona información útil para tomar decisiones sobre compras, reposición y administración de productos.

6.7. MÓDULO DE CENTRO DE AYUDA

Archivo principal: `src/pages/Ayuda.jsx`.

Este módulo presenta instrucciones para utilizar las funcionalidades principales del sistema.

Incluye guías para:

- Registrar productos.
- Ajustar existencias.
- Comprender las alertas.
- Revisar los movimientos.
- Administrar categorías.
- Descargar informes.

Importancia: Facilita el aprendizaje y permite que el usuario conozca cómo utilizar correctamente la aplicación.

7. LÓGICA DE PROGRAMACIÓN Y CÁLCULOS PRINCIPALES

Durante el desarrollo se utilizaron diferentes métodos y funciones de JavaScript.

Método o función	Aplicación
`useState()`	Administra los datos y estados de la aplicación.
`useMemo()`	Memoriza resultados de operaciones de filtrado.
`useEffect()`	Reinicia los datos del control cuando cambia el producto seleccionado.
`map()`	Actualiza productos y categorías sin modificar directamente el arreglo original.
`filter()`	Busca productos y selecciona movimientos.
`find()`	Localiza un producto por su identificador.

`some()`	Comprueba si existen códigos o categorías duplicados.
`reduce()`	Calcula cantidades y valores económicos.
`sort()`	Ordena los productos según su valor.
`Math.max()`	Evita que el stock sea negativo.

7.1. Cálculo del valor total

La función `valorTotal()` utiliza `reduce()` para sumar el precio multiplicado por la cantidad disponible de cada producto.

Ejemplo:

Si un producto cuesta S/ 20 y existen 10 unidades:

20 × 10 = S/ 200.

Este resultado representa el valor de ese producto dentro del inventario.

7.2. Clasificación del estado de los productos

La función `estado()` determina el estado según las existencias:

- Disponible: cuando el stock es igual o superior al mínimo establecido.
- Bajo mínimo: cuando el stock es mayor que cero, pero menor que el mínimo.
- Agotado: cuando el stock es igual a cero.

Esta clasificación permite generar alertas y facilitar la reposición.

7.3. Cálculo de reposición

Para determinar las unidades que faltan se aplica:

Unidades por reponer = Stock mínimo − Stock actual.

Luego:

Costo de reposición = Unidades por reponer × Precio unitario.

Estos cálculos permiten estimar cuánto costaría recuperar el nivel mínimo de existencias.

8. FUNCIONAMIENTO GENERAL DEL SISTEMA

El sistema funciona mediante los siguientes procedimientos:

1. El usuario ejecuta la aplicación en el navegador.
2. Aparece el módulo principal de Inventario.
3. El usuario puede consultar los productos registrados.

4. Puede agregar un nuevo producto mediante el formulario.
5. El sistema valida los datos antes de guardar.
6. Se actualiza la lista de productos.
7. El usuario puede registrar entradas o salidas de existencias.
8. El sistema actualiza las cantidades y registra los movimientos.
9. Los módulos de Categorías e Informes presentan información organizada.
10. Finalmente, el usuario puede exportar los datos para consultarlos externamente.

El componente `App.jsx` coordina las operaciones y comparte la información con los diferentes módulos.

9. INSTALACIÓN Y EJECUCIÓN DEL PROYECTO

Para ejecutar el proyecto se necesita tener instalado Node.js y contar con un editor de código como Visual Studio Code.

Procedimiento de instalación:

1. Descomprimir el archivo ZIP del proyecto.
2. Abrir la carpeta `proyecto-02-control-inventario` en Visual Studio Code.
3. Abrir una terminal dentro de la carpeta.
4. Instalar las dependencias con el siguiente comando:

```
npm install
```

Este comando instala las bibliotecas necesarias, entre ellas React, React DOM, Vite y Lucide React.

5. Ejecutar el proyecto:

```
npm run dev
```

6. Esperar a que Vite muestre la dirección local en la terminal.
7. Abrir esa dirección en el navegador para visualizar el sistema.

Comprobación de compilación:

También se puede ejecutar:

```
npm run build
```

Este comando permite comprobar que el proyecto pueda compilarse y generar los archivos optimizados para producción.

10. PRUEBAS Y VALIDACIONES DEL SISTEMA

Para comprobar el funcionamiento se consideran las siguientes pruebas:

Prueba	Resultado esperado

Registrar un producto con datos completos	El producto se agrega correctamente.
Registrar un código duplicado	El sistema muestra un mensaje de error.
Registrar un precio negativo	El sistema rechaza el valor.
Registrar cantidades decimales	El sistema rechaza el stock no entero.
Registrar una entrada	Aumenta el stock y se registra el movimiento.
Registrar una salida válida	Disminuye el stock y se registra el movimiento.
Registrar una salida mayor al stock disponible	El sistema impide la operación.
Eliminar una categoría con productos	No permite eliminarla.
Buscar un producto	Muestra los resultados coincidentes.
Descargar el informe	Genera un archivo CSV.

Estas pruebas permiten verificar las reglas de validación y las operaciones principales implementadas en el código.

11. LIMITACIONES Y MEJORAS FUTURAS

El proyecto utiliza actualmente el estado de React para almacenar los productos, categorías y movimientos en memoria.

Esto significa que los cambios realizados durante la sesión no se conservan después de recargar la página o cerrar la aplicación.

También presenta las siguientes limitaciones:

- No cuenta con una base de datos permanente.
- No tiene un sistema de autenticación de usuarios.
- No dispone de diferentes permisos para administradores y trabajadores.
- El historial de movimientos se conserva únicamente durante la sesión.
- No cuenta con sincronización entre diferentes dispositivos.

Mejoras propuestas:

- Implementar una base de datos MySQL o PostgreSQL.
- Incorporar un sistema de inicio de sesión.
- Establecer roles y permisos de usuario.
- Crear reportes descargables en PDF.
- Implementar copias de seguridad.
- Incorporar un historial permanente de movimientos.
- Permitir el acceso simultáneo desde diferentes dispositivos.

12. CONCLUSIONES

1. Se desarrolló una aplicación web que permite administrar productos y controlar las existencias mediante diferentes módulos.
2. React permitió organizar la aplicación en componentes reutilizables, mientras que JavaScript permitió implementar las operaciones de registro, edición, eliminación y control de stock.
3. Las validaciones implementadas ayudan a prevenir errores como códigos duplicados, precios negativos y salidas superiores a las existencias disponibles.
4. Los cálculos automáticos permiten conocer el valor económico del inventario, identificar productos que necesitan reposición y generar informes.
5. El proyecto demuestra la aplicación práctica de herramientas de desarrollo web para resolver una necesidad de gestión comercial.
6. Para mejorar el sistema en el futuro, será necesario incorporar una base de datos que permita conservar la información permanentemente.

13. REFERENCIAS TÉCNICAS

- React: https://react.dev/
- Vite: https://vite.dev/
- JavaScript: https://developer.mozilla.org/es/docs/Web/JavaScript
- Lucide React: https://lucide.dev/guide/packages/lucide-react
- CSS: https://developer.mozilla.org/es/docs/Web/CSS